

DSA MASTER SHEET

Volume 2 — Advanced Patterns & Company Rounds

Segment Trees · Advanced Graphs · Digit DP · String Algorithms · Number Theory

Company-tagged: Google · Meta · Amazon · Microsoft · Apple · Netflix

Volume 2 covers	<i>Advanced DSA topics not in Vol 1: Segment Trees, Suffix Arrays, Digit DP, Concurrency, and all company-tagged favorites.</i>
Complexity shown	<i>Every pattern now includes Time & Space complexity so you never blank during interviews.</i>
Problem notes	<i>Each problem has a short hint in italics — use only after a 15-min attempt.</i>
Combined study plan	<i>Vol 1 (90 days) + Vol 2 (60 days) = complete MAANG-ready preparation.</i>

Table of Contents

01. Segment Trees & Fenwick Trees

Fenwick Tree (BIT) — Point Update, Range Query · Segment Tree — Range Update, Range Query · Sparse Table — Static RMQ

02. Advanced Graph Algorithms

Minimum Spanning Tree (Prim's & Kruskal's) · Strongly Connected Components (Kosaraju / Tarjan) · Floyd-Warshall — All Pairs Shortest Path · Bipartite Matching & Network Flow · Euler Path / Hamiltonian Path

03. Advanced Dynamic Programming

Digit DP · Bitmask DP · DP on Strings — Advanced · Probability / Expected Value DP · Convex Hull Trick / Divide and Conquer DP

04. String Algorithms

KMP — Pattern Matching · Rabin-Karp / Rolling Hash · Z-Algorithm · Suffix Array & LCP Array · Palindrome Structures (Manacher's)

05. Math & Number Theory

GCD, LCM & Euclidean Algorithm · Prime Sieve (Sieve of Eratosthenes) · Modular Arithmetic & Fast Exponentiation · Combinatorics & Counting · Geometry & Coordinate Math

06. Intervals & Sweep Line

Merge & Insert Intervals · Sweep Line · Interval DP

07. Design & Simulation

LRU / LFU Cache Design · Monotonic Stack / Queue Design · Iterator Design · Randomized Data Structures

08. Two Pointers & Sorting — Deep Dive

K-Sum Generalization · Counting Inversions & Merge Sort · Cyclic Sort Pattern · Quickselect (K-th Element)

09. Concurrency & System-Level Problems

Semaphore / Mutex Patterns · Producer-Consumer / Reader-Writer

10. Mixed / Hard Company-Tagged Problems

Google Favorites · Meta (Facebook) Favorites · Amazon Favorites · Microsoft Favorites · Apple & Netflix (Emerging Patterns)

Segment Trees & Fenwick Trees

For range queries (sum, min, max, GCD) with point/range updates in $O(\log n)$. Essential for competitive DSA rounds.

Pattern: Fenwick Tree (BIT) — Point Update, Range Query

Tip: Simplest range sum structure. Build in $O(n \log n)$. `update()` and `query()` both $O(\log n)$.

Time: $O(\log n)$ per op | Space: $O(n)$

[Medium] Range Sum Query — Mutable

→ Classic BIT intro

[Hard] Count of Smaller Numbers After Self

→ BIT + coordinate compression

[Hard] Reverse Pairs

→ BIT + merge sort hybrid

[Medium] Number of Longest Increasing Subsequences

→ BIT on LIS length

[Hard] Create Sorted Array Through Instructions

→ BIT counting inversions

[Hard] Sum of Prefix Scores of Strings

→ BIT on trie

Pattern: Segment Tree — Range Update, Range Query

Tip: Use lazy propagation for range updates. Essential for interval modification problems.

Time: $O(\log n)$ per op | Space: $O(4n)$

[Medium] Range Minimum Query

→ Build and query segment tree

[Hard] My Calendar III (max k-booking)

→ Segment tree with lazy prop

[Hard] The Skyline Problem

→ Segment tree on events

[Hard] Falling Squares

→ Coordinate compress + seg tree

[Hard] Count of Range Sum

→ Merge sort or seg tree

[Hard] Minimum Number of Increments on Subarrays

→ Difference array + seg tree

[Hard] Rectangle Area II

→ Coordinate compress + seg tree

Pattern: Sparse Table — Static RMQ

Tip: For static arrays with range min/max queries. $O(1)$ query after $O(n \log n)$ build.

Time: $O(1)$ query, $O(n \log n)$ build | Space: $O(n \log n)$

[Medium] Range Minimum Query (static)

→ Classic sparse table use case

[Medium] Longest Turbulent Subarray

→ Sparse table on range

[Medium] Maximum Width Ramp

→ Monotone + sparse table

Advanced Graph Algorithms

Beyond BFS/DFS. These patterns appear in senior-level MAANG rounds and design interviews.

Pattern: Minimum Spanning Tree (Prim's & Kruskal's)

Tip: Kruskal = sort edges + union-find. Prim = greedy with min-heap from a source.

Kruskal: $O(E \log E)$ | **Prim:** $O(E \log V)$

[Medium] Min Cost to Connect All Points

→ Prim's on dense graph

[Hard] Optimize Water Distribution in a Village

→ MST with virtual node

[Hard] Find Critical and Pseudo-Critical Edges in MST

→ Kruskal + edge classification

[Medium] Connecting Cities With Minimum Cost

→ Kruskal with union-find

[Medium] Minimum Spanning Tree (weighted)

→ Classic Kruskal

Pattern: Strongly Connected Components (Kosaraju / Tarjan)

Tip: Tarjan's uses low-link values in DFS. SCCs help identify cycles and condensation DAGs.

Time: $O(V + E)$ | **Space:** $O(V)$

[Hard] Number of Strongly Connected Components

→ Kosaraju's algorithm

[Hard] Critical Connections in a Network (Bridges)

→ Tarjan bridge-finding

[Hard] Articulation Points in a Graph

→ Tarjan articulation points

[Hard] Redundant Connection II

→ SCC + union find

[Medium] Course Schedule (cycle in directed graph)

→ DFS + coloring

Pattern: Floyd-Warshall — All Pairs Shortest Path

Tip: Triple nested loop $dp[k][i][j]$. Use when $V \leq 300$. Detect negative cycles via $dp[i][i] < 0$.

Time: $O(V^3)$ | Space: $O(V^2)$

[Medium] Find the City with Smallest Number of Neighbors

→ Floyd-Warshall + filter

[Medium] Cheapest Flights Within K Stops

→ Bellman-Ford variant

[Medium] Network Delay Time

→ Compare Dijkstra vs Floyd

[Medium] Evaluate Division

→ Floyd on ratio graph

[Hard] Shortest Path Visiting All Nodes

→ BFS + bitmask DP

Pattern: Bipartite Matching & Network Flow

Tip: Bipartite check via 2-coloring BFS. Max matching via augmenting paths (Hopcroft-Karp).

Bipartite check: $O(V+E)$ | Max Flow: $O(VE^2)$

[Medium] Is Graph Bipartite?

→ 2-color BFS/DFS

[Medium] Possible Bipartition

→ Bipartite check on people graph

[Hard] Maximum Bipartite Matching

→ Hungarian or Hopcroft-Karp

[Medium] Flower Planting With No Adjacent

→ Graph coloring

[Hard] Minimum Cost Flow (assignment problem)

→ Flow network modeling

Pattern: Euler Path / Hamiltonian Path

Tip: Euler path exists if 0 or 2 odd-degree vertices. Use Hierholzer's algorithm.

Euler: $O(E)$ | Hamiltonian: $O(n!)$ brute / $O(2^n \cdot n)$ DP

[Hard] Reconstruct Itinerary

→ Euler path in directed graph

[Hard] Valid Arrangement of Pairs

→ Eulerian path on pairs

[Hard] Cracking the Safe

→ De Bruijn sequence / Euler

[Medium] Find Minimum Number of Vertices to Reach All Nodes

→ In-degree = 0 greedy

Advanced Dynamic Programming

These DP patterns separate good candidates from great ones. Expect them in L5/L6 Google & Meta loops.

Pattern: Digit DP

Tip: Count numbers in $[L, R]$ satisfying a property. State: (position, tight, sum/digit_count).

Time: $O(D * \text{states})$ | **Space:** $O(D * \text{states})$ where $D = \text{digits}$

[Medium] Count Numbers with Unique Digits

→ Digit DP intro

[Hard] Non-negative Integers Without Consecutive Ones

→ Digit DP + binary

[Hard] Numbers At Most N Given Digit Set

→ Digit DP with set

[Hard] Digit Count in Range

→ Count digit d in $[lo, hi]$

[Medium] Rotated Digits

→ Digit DP + validity check

[Hard] Count Special Integers

→ Digit DP + used-digits bitmask

Pattern: Bitmask DP

*Tip: State = bitmask of visited/chosen elements. Classic: TSP, assignment. $O(2^n * n)$.*

Time: $O(2^n * n)$ | **Space:** $O(2^n * n)$

[Hard] Shortest Path Visiting All Nodes

→ BFS + bitmask

[Medium] Partition to K Equal Sum Subsets

→ Bitmask DP backtrack

[Hard] Maximum AND Sum of Array

→ Bitmask DP assignment

[Hard] Minimum XOR Sum of Two Arrays

→ Bitmask DP matching

[Hard] Distribute Repeating Integers

→ Bitmask DP over customers

[Hard] Find the Shortest Superstring

→ TSP-style bitmask DP

Pattern: DP on Strings — Advanced

Tip: Combine hashing, rolling hash, or Z-function with DP for string pattern problems.

Time: $O(n^2)$ or $O(n^2 * m)$ | Space: $O(n^2)$

[Hard] Distinct Subsequences

→ 2D DP counting matches

[Hard] Regular Expression Matching

→ DP with . and * patterns

[Hard] Wildcard Matching

→ DP with ? and * patterns

[Medium] Interleaving String

→ 2D DP path

[Hard] Edit Distance

→ Levenshtein distance DP

[Hard] Shortest Common Supersequence

→ LCS + reconstruction

[Medium] Minimum ASCII Delete Sum for Two Strings

→ Weighted LCS variant

Pattern: Probability / Expected Value DP

Tip: $E[X] = \text{sum over states of (prob * value)}$. Often involves fraction arithmetic or floating point.

Time: $O(n * \text{states})$ | Space: $O(\text{states})$

[Medium] Knight Probability in Chessboard

→ Probability DP on grid

[Medium] New 21 Game

→ Sliding window + probability DP

[Medium] Soup Servings

→ Memoized probability DP

[Medium] Airplane Seat Assignment Probability

→ Math + DP insight

[Medium] Dice Roll Simulation

→ Constrained count DP

Pattern: Convex Hull Trick / Divide and Conquer DP

Tip: CHT reduces $O(n^2)$ DP to $O(n \log n)$ when cost is linear. D&C; DP when opt is monotone.

CHT: $O(n \log n)$ | D&C; DP: $O(n \log n)$

[Hard] Minimum Cost to Cut a Stick

→ Interval DP, D&C; opt

[Hard] Burst Balloons

→ Interval DP classic

[Hard] Stone Game V

→ Interval DP

[Hard] Minimum Number of Refueling Stops

→ Greedy + heap or DP

[Medium] Largest Divisible Subset

→ LIS-style DP

String Algorithms

Pattern matching, suffix structures, and hashing. Appear in L4+ Google, Amazon, and Bloomberg interviews.

Pattern: KMP — Pattern Matching

Tip: Build failure function ($lps[]$) in $O(m)$. Use it to skip comparisons. Total $O(n+m)$.

Time: $O(n + m)$ | Space: $O(m)$

[Easy] Find the Index of First Occurrence in String

→ strStr — implement KMP

[Hard] Shortest Palindrome

→ KMP on $s + \text{rev}(s)$

[Easy] Repeated Substring Pattern

→ KMP failure function insight

[Medium] Minimum Period of a String

→ KMP period = $n - lps[n-1]$

[Medium] String Matching in an Array

→ KMP for substring check

Pattern: Rabin-Karp / Rolling Hash

Tip: Hash substrings in $O(1)$ after $O(n)$ preprocessing. Detect duplicates / match patterns.

Time: $O(n + m)$ average | Space: $O(1)$

[Hard] Longest Duplicate Substring

→ Binary search + rolling hash

[Medium] Repeated DNA Sequences

→ Rolling hash on 4-char alphabet

[Medium] Find All Anagrams in a String

→ Hash comparison sliding window

[Hard] Longest Happy Prefix

→ KMP or rolling hash

[Hard] Distinct Echo Substrings

→ Rolling hash + DP

Pattern: Z-Algorithm

Tip: $z[i]$ = length of longest substring starting at i matching prefix. $O(n)$ pattern matching.

Time: $O(n)$ | Space: $O(n)$

[Medium] Count Occurrences of Pattern in Text

→ Z-array basic

[Medium] Minimum Characters to Add at Front to Make Palindrome

→ Z + reverse

[Medium] Sum of Beauty of All Substrings

→ Z-function application

[Easy] Check if String is a Rotation of Another

→ Z on doubled string

Pattern: Suffix Array & LCP Array

Tip: Sort all suffixes. $LCP[i]$ = common prefix between $suffix[i]$ and $suffix[i-1]$. SA-IS in $O(n)$.

Time: $O(n \log n)$ naive, $O(n)$ SA-IS | Space: $O(n)$

[Hard] Longest Duplicate Substring

→ Suffix array + binary search

[Hard] Number of Distinct Substrings

→ SA + LCP formula: $n*(n+1)/2 - \text{sum}(LCP)$

[Hard] Longest Common Substring of Two Strings

→ SA on concatenated string

[Hard] Minimum Suffix Array

→ Build and query

Pattern: Palindrome Structures (Manacher's)

Tip: Manacher's finds all palindromic substrings in $O(n)$. Expand around center is $O(n^2)$.

Time: $O(n)$ Manacher | Space: $O(n)$

[Medium] Longest Palindromic Substring

→ Manacher's or expand-around-center

[Medium] Palindromic Substrings (count)

→ Manacher's count

[Hard] Palindrome Pairs

→ Trie + reversal check

[Medium] Longest Palindrome by Concatenating Two Letter Words

→ Greedy + hash

[Hard] Count Palindromic Subsequences

→ DP + Manacher insight

Math & Number Theory

GCD, primes, modular arithmetic, combinatorics. Appear in Google and Jane Street rounds.

Pattern: GCD, LCM & Euclidean Algorithm

*Tip: $\gcd(a, b) = \gcd(b, a \% b)$. $\text{lcm}(a, b) = a * b // \gcd(a, b)$. Extended Euclidean for modular inverse.*

Time: $O(\log \min(a, b))$

[Easy] GCD of Strings

→ gcd on string lengths

[Medium] Fraction Addition and Subtraction

→ GCD for simplification

[Medium] Minimum Operations to Make Array Equal II

→ GCD constraint check

[Medium] Water and Jug Problem

→ Bezout identity + GCD

[Medium] Check If Array Pairs Are Divisible by K

→ Modular arithmetic

[Hard] Count Array Pairs Divisible by K

→ GCD + frequency map

Pattern: Prime Sieve (Sieve of Eratosthenes)

Tip: Generate all primes up to N in $O(N \log \log N)$. Use for prime factorization problems.

Time: $O(N \log \log N)$ | Space: $O(N)$

[Medium] Count Primes

→ Classic sieve

[Medium] Closest Divisors

→ Factorization search

[Medium] 4 Keys Keyboard

→ DP + prime-like growth

[Medium] Smallest Value After Replacing with Sum of Prime Factors

→ Factorize + replace

[Medium] Product of Prime Factors

→ Sieve + DP

Pattern: Modular Arithmetic & Fast Exponentiation

Tip: Always take mod during computation, not at the end. Fast pow: $O(\log n)$ via repeated squaring.

Fast pow: $O(\log n)$ | Mod inverse: $O(\log \text{MOD})$

[Medium] Pow(x, n)

→ Fast exponentiation

[Medium] Count Vowels Permutation

→ Matrix exp or DP + mod

[Medium] Unique Paths (combinatorics)

→ $C(m+n-2, m-1) \bmod p$

[Medium] Catalan Number

→ Modular binomial

[Medium] Super Pow

→ Modular exponentiation

[Medium] Fibonacci Number (fast)

→ Matrix exponentiation $O(\log n)$

Pattern: Combinatorics & Counting

Tip: Precompute factorials and inverse factorials for nCr in $O(1)$. Stars and bars for distribution.

Precompute: $O(n)$ | nCr query: $O(1)$

[Medium] Unique Binary Search Trees

→ Catalan numbers

[Hard] K-th Smallest in Lexicographic Order

→ Combinatorial counting

[Medium] Count Sorted Vowel Strings

→ Stars and bars

[Hard] Number of Ways to Wear Different Hats

→ Bitmask + combinatorics

[Hard] Handshakes That Don't Cross

→ Catalan number

[Medium] Count Good Numbers

→ Fast exponent + mod

Pattern: Geometry & Coordinate Math

Tip: Use cross product for orientation. Avoid floating point: scale to integers.

Per query: $O(1)$ to $O(n \log n)$

[Hard] Max Points on a Line

→ Slope as fraction + hash

[Medium] Minimum Area Rectangle

→ Hash + diagonal pairs

[Hard] Convex Hull (Graham Scan)

→ Sort + cross product

[Medium] K Closest Points to Origin

→ Distance comparison

[Easy] Rectangle Area Overlap

→ Coordinate intersection

[Hard] Check If Two Line Segments Intersect

→ Cross product orientation

Intervals & Sweep Line

Model events as points on a timeline. Sort by start or coordinate. Essential for scheduling and geometry.

Pattern: Merge & Insert Intervals

Tip: Sort by start. Merge when $cur.start \leq prev.end$. Track $end = \max(end, cur.end)$.

Time: $O(n \log n)$ | Space: $O(n)$

[Medium] Merge Intervals

→ Sort + greedy merge

[Medium] Insert Interval

→ Three-phase insert

[Medium] Non-overlapping Intervals (min removal)

→ Greedy sort by end

[Medium] Meeting Rooms II (min rooms)

→ Min-heap or sweep

[Hard] Employee Free Time

→ Merge across lists

[Medium] Remove Covered Intervals

→ Sort by start, desc by end

Pattern: Sweep Line

Tip: Add +1 at start, -1 at end+1. Sort events by coordinate. Running sum = active count.

Time: $O(n \log n)$ | Space: $O(n)$

[Hard] The Skyline Problem

→ Max-heap + sweep events

[Hard] My Calendar III

→ Difference array or segment tree

[Medium] Minimum Number of Arrows to Burst Balloons

→ Sort end + greedy

[Medium] Count Number of Texts

→ Sweep + DP

[Hard] Describe the Painting

→ Coordinate compress + sweep

[Hard] Amount of New Area Painted Each Day

→ Sweep + set of unpainted

Pattern: Interval DP

Tip: Split range $[i, j]$ at every k . $dp[i][j] = \text{best combination of } dp[i][k] \text{ and } dp[k+1][j]$.

Time: $O(n^3)$ | Space: $O(n^2)$

[Hard] Burst Balloons

→ Classic interval DP

[Hard] Strange Printer

→ Interval DP on characters

[Hard] Minimum Cost to Merge Stones

→ Generalized stone game

[Hard] Zuma Game

→ Interval DP on groups

[Hard] Remove Boxes

→ 3D interval DP

[Medium] Guess Number Higher or Lower II

→ Minimax interval DP

Design & Simulation

System design in code form. Test your ability to use the right data structure for the right job.

Pattern: LRU / LFU Cache Design

Tip: LRU = doubly linked list + hashmap. LFU adds frequency buckets (map of freq -> list).

LRU: $O(1)$ all ops | LFU: $O(1)$ all ops

[Medium] LRU Cache

→ Classic hashmap + DLL

[Hard] LFU Cache

→ Freq buckets + hashmap

[Hard] All O'one Data Structure

→ DLL + two hashmaps

[Medium] Design Twitter

→ Heap + user follow graph

[Hard] Design In-Memory File System

→ Trie + directory map

Pattern: Monotonic Stack / Queue Design

Tip: Design structures that answer 'max/min of window' or 'next greater' in $O(1)$ amortized.

Amortized $O(1)$ per operation

[Medium] Min Stack

→ Pair stack trick

[Hard] Max Stack

→ Doubly linked list + sorted map

[Medium] Design Circular Queue

→ Array with front/rear pointers

[Medium] Design Hit Counter

→ Deque + sliding window

[Hard] Sliding Window Maximum

→ Monotonic deque

Pattern: Iterator Design

Tip: Lazy evaluation — only compute next element on demand. Use stack for nested structures.

hasNext/next: $O(1)$ amortized

[Medium] Flatten Nested List Iterator

→ Stack-based lazy flatten

[Medium] Flatten 2D Vector

→ Pointer pair iteration

[Medium] Zigzag Iterator

→ Round-robin iterator

[Medium] Peeking Iterator

→ Wrapper with lookahead

[Medium] Binary Search Tree Iterator

→ Inorder lazy DFS

Pattern: Randomized Data Structures

Tip: Reservoir sampling for streaming. Shuffle = Fisher-Yates. getRandom in $O(1)$ needs array + map.

$O(1)$ per operation with array+hashmap

[Medium] Insert Delete GetRandom $O(1)$

→ Array + hashmap swap trick

[Hard] Insert Delete GetRandom $O(1)$ — Duplicates

→ Map to set of indices

[Medium] Random Pick with Weight

→ Prefix sum + binary search

[Medium] Shuffle an Array

→ Fisher-Yates

[Medium] Random Pick Index

→ Reservoir sampling

[Medium] Linked List Random Node

→ Reservoir sampling on list

Two Pointers & Sorting — Deep Dive

Many array problems reduce to sorting + a smart traversal. Master these patterns cold.

Pattern: K-Sum Generalization

Tip: Reduce K-sum to (K-2)-sum recursively using sorted array + two pointers. Skip duplicates!

Time: $O(n^{k-1})$ | **Space:** $O(k)$ recursion

[Easy] Two Sum

→ Hash map $O(n)$

[Medium] 3Sum

→ Fix one, two-pointer rest

[Medium] 3Sum Closest

→ Track min abs diff

[Medium] 4Sum

→ Fix two, two-pointer rest

[Medium] 4Sum II

→ Two hashmaps split

[Hard] K-Sum Count (general)

→ Recursive K-sum

Pattern: Counting Inversions & Merge Sort

Tip: Inversions count = number of (i, j) pairs. Modified merge sort counts during merge in $O(n \log n)$.

Time: $O(n \log n)$ | **Space:** $O(n)$

[Hard] Count of Smaller Numbers After Self

→ Merge sort inversion count

[Hard] Reverse Pairs

→ Merge sort with threshold

[Medium] Count Inversions in Array

→ Classic merge sort

[Medium] Global and Local Inversions

→ Merge sort + max prefix

[Medium] Sort Array by Parity II

→ Two-pointer partition

Pattern: Cyclic Sort Pattern

Tip: When values in $[1, n]$ or $[0, n-1]$, place each value at its correct index. Find missing/duplicates in $O(n)$.

Time: $O(n)$ | Space: $O(1)$

[Easy] Find the Missing Number

→ Cyclic sort or XOR

[Medium] Find All Missing Numbers

→ Cyclic sort

[Medium] Find the Duplicate Number

→ Cyclic sort or Floyd's

[Medium] Find All Duplicates

→ Negate index trick

[Hard] First Missing Positive

→ Cyclic sort on positives

[Easy] Set Mismatch

→ Cyclic sort find swap

Pattern: Quickselect (K-th Element)

Tip: Average $O(n)$ using partition. Worst case $O(n^2)$ — use median-of-medians for $O(n)$ guarantee.

Average $O(n)$ | Worst $O(n^2)$

[Medium] Kth Largest Element in an Array

→ Quickselect or heap

[Medium] Kth Smallest Element in a Sorted Matrix

→ Binary search or heap

[Hard] Wiggle Sort II

→ Quickselect + index trick

[Medium] Top K Frequent Elements

→ Bucket sort $O(n)$ variant

[Medium] Find K-th Largest XOR Coordinate Value

→ Prefix XOR + quickselect

Concurrency & System-Level Problems

Threading problems appear in Amazon & Microsoft OS rounds. Use semaphores, mutexes, and barriers.

Pattern: Semaphore / Mutex Patterns

Tip: Semaphore = counter-based lock. Use `acquire()` / `release()` to coordinate threads.

Depends on thread count and lock contention

[Easy] Print in Order

→ Two semaphores chain

[Medium] Print FooBar Alternately

→ Two semaphores alternating

[Medium] Print Zero Even Odd

→ Three semaphores with roles

[Medium] Building H₂O

→ Barrier pattern for H₂O

[Medium] The Dining Philosophers

→ Deadlock avoidance with locks

[Medium] Fizz Buzz Multithreaded

→ Four threads with conditions

Pattern: Producer-Consumer / Reader-Writer

Tip: Classic OS patterns. Use condition variables for signaling between thread roles.

Lock-based: O(1) per op with contention overhead

[Medium] Design Bounded Blocking Queue

→ Condition variable + deque

[Medium] Web Crawler Multithreaded

→ BFS with thread pool

[Medium] Traffic Light Controlled Intersection

→ Mutex + turn tracking

Mixed / Hard Company-Tagged Problems

Frequently asked in final rounds at Google, Meta, Amazon, Apple, Netflix. These need 2-3 pattern combinations.

Pattern: Google Favorites

Tip: Google loves segment trees, interval problems, math reasoning, and hard graph problems.

Varies

[Hard] Minimum Window Substring

→ Sliding window + freq map

[Hard] Word Ladder II

→ BFS + backtrack all paths

[Hard] Alien Dictionary

→ Topological sort

[Hard] Largest Rectangle in Histogram

→ Monotonic stack

[Hard] Text Justification

→ Greedy string formatting

[Hard] Trapping Rain Water II (3D)

→ BFS min-heap

[Hard] The Skyline Problem

→ Sweep + max-heap

[Hard] Count of Range Sum

→ Merge sort / BIT

[Hard] Median of Two Sorted Arrays

→ Binary search halves

[Hard] Maximum Vacation Days

→ DP on cities

Pattern: Meta (Facebook) Favorites

Tip: Meta favors graph traversal, string manipulation, and real-world system modeling.

Varies

- [Medium]** Clone Graph
 - BFS/DFS + hashmap
- [Hard]** Expression Add Operators
 - Backtracking + eval
- [Hard]** Remove Invalid Parentheses
 - BFS / backtrack all valid
- [Medium]** Accounts Merge
 - Union-Find + sorting
- [Medium]** Minimum Remove to Make Valid Parentheses
 - Two-pass stack
- [Medium]** Random Pick with Weight
 - Prefix sum + rand
- [Medium]** Add Bold Tag in String
 - Interval merging
- [Medium]** Buildings With an Ocean View
 - Monotonic stack
- [Medium]** Subarray Sum Equals K
 - Prefix sum + hashmap
- [Medium]** Dot Product of Two Sparse Vectors
 - HashMap compression

Pattern: Amazon Favorites

Tip: Amazon loves BFS/DFS, heaps, DP, and real-world logistics modeling.

Varies

- [Medium]** Number of Islands
 - BFS/DFS grid
- [Medium]** Reorder Data in Log Files
 - Custom sort + stable
- [Medium]** Top K Frequent Words
 - Heap + freq map
- [Hard]** Merge K Sorted Lists
 - Min-heap merging
- [Medium]** Longest Palindromic Substring
 - Expand or Manacher
- [Easy-Hard]** Two Sum (all variants)
 - Hash / sort / binary search
- [Medium]** LRU Cache
 - DLL + hashmap
- [Medium]** K Closest Points to Origin
 - Quickselect or heap
- [Easy]** Flood Fill
 - BFS/DFS grid coloring
- [Medium]** Optimal Utilization (pair sum closest to target)
 - Sort + two pointers

Pattern: Microsoft Favorites

Tip: Microsoft focuses on trees, recursion, and linked-list manipulation in onsite rounds.

Varies

[Easy] Reverse Linked List

→ Iterative + recursive

[Medium] Lowest Common Ancestor (BT)

→ Postorder DFS

[Hard] Serialize and Deserialize Binary Tree

→ BFS encoding

[Easy] Valid Parentheses

→ Stack matching

[Medium] Find All Anagrams in a String

→ Sliding window freq

[Medium] Word Search

→ DFS + backtrack grid

[Medium] Binary Tree Right Side View

→ BFS level order last

[Easy] Maximum Depth of Binary Tree

→ Recursive DFS

[Medium] Spiral Matrix

→ Direction simulation

[Medium] Jump Game

→ Greedy max reach

Pattern: Apple & Netflix (Emerging Patterns)

Tip: Apple leans toward low-level logic and math. Netflix favors streaming/media-inspired DP and greedy.

Varies

[Medium] Find the Duplicate Number

→ Floyd's cycle detection

[Medium] Integer to Roman / Roman to Integer

→ Greedy + map

[Medium] Valid Sudoku

→ Hash set validation

[Medium] Decode Ways

→ 1D DP with constraints

[Medium] Minimum Path Sum

→ 2D DP

[Medium] Meeting Rooms II

→ Min-heap scheduling

[Hard] Find Median from Data Stream

→ Two heaps

[Hard] Sliding Window Maximum

→ Monotonic deque

[Hard] Minimum Cost to Hire K Workers

→ Ratio sort + heap

[Medium] Course Schedule II

→ Topological sort

Combined 150-Day Study Roadmap

Days 1–15	Arrays, Strings, Hashing <i>Two pointers, sliding window, prefix sum, frequency maps</i>
Days 16–30	Linked Lists, Stacks, Queues <i>Reversal, fast/slow pointers, monotonic stack</i>
Days 31–45	Trees & BST, Heaps <i>DFS/BFS trees, BST properties, top-K, two-heap median</i>
Days 46–60	Graphs I <i>BFS/DFS, union-find, topological sort</i>
Days 61–75	Recursion, Backtracking, Binary Search <i>Subsets, permutations, binary search on answer</i>
Days 76–90	Dynamic Programming I <i>1D DP, 2D DP, knapsack, LIS/LCS</i>
Days 91–100	Advanced Graphs <i>MST, SCC, Floyd-Warshall, bipartite, Euler path</i>
Days 101–110	Advanced DP <i>Digit DP, bitmask DP, probability DP, CHT</i>
Days 111–118	String Algorithms <i>KMP, Z-algo, Rabin-Karp, Manacher, Suffix Array</i>
Days 119–125	Segment Trees, BIT, Sparse Table <i>Range queries, lazy propagation, BIT inversions</i>
Days 126–132	Math & Number Theory <i>Primes, GCD, modular arithmetic, combinatorics</i>

Days 133–138

Intervals, Sweep Line, Design

LRU/LFU, sweep events, iterator design

Days 139–150

Company Mock Rounds

Google, Meta, Amazon, Microsoft, Apple, Netflix tagged sets

Volume 1 + Volume 2 combined = 300+ problems, 70+ patterns, 12 core + 8 advanced topics. You now have everything needed to pass any MAANG technical interview. Trust the process, be consistent, and review patterns — not just solutions.